

МИНОБРНАУКИ РОССИИ  
Федеральное государственное бюджетное образовательное учреждение  
высшего образования  
«ВЛАДИВОСТОКСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»  
(ФГБОУ ВО «ВВГУ»)

Институт информационных технологий и анализа данных  
Кафедра информационных технологий и систем

ОТЧЁТ ПО УЧЕБНОЙ ОЗНАКОМИТЕЛЬНОЙ  
ПРАКТИКЕ

Оценка удов.

Студент  
гр. БИС-24-3



М. Н. Жиликов

Руководитель,  
Канд. тех. наук



И. С. Можаровский

618. удов.

МИНОБРНАУКИ РОССИИ  
Федеральное государственное бюджетное образовательное учреждение высшего  
образования  
«ВЛАДИВОСТОКСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»  
(ФГБОУ ВО «ВВГУ»)

Институт информационных технологий и анализа данных Кафедра  
информационных технологий и систем

Индивидуальное задание  
на учебную ознакомительную практику

Студенту гр. БИС-24-3 Жияков Максим Николаевич

1 Разработать программный продукт «Визуализация основных структур данных» (вариант А-04):

- Реализовать стек (Stack), очередь (Queue) и двоичное дерево поиска (BST).
- Пошаговая визуализация операций: push/pop для стека, enqueue/dequeue для очереди, insert/delete/search для BST.
- Отображение состояния структуры данных после каждой операции.
- Журнал операций с возможностью отмены (Undo).
- Сравнение временной сложности операций в таблице.

2 Соблюдать общие требования к выполнению задания (документ «Варианты индивидуального задания», раздел 2):

- Выбор языка программирования с обоснованием.
- Хранение данных в файлах без СУБД.
- Контейнеризация (Dockerfile).
- Тестирование (юнит- и интеграционные тесты, тестовые данные).
- Отчёт 20–25 страниц по СК-СТО-ТР-04-1.005-2015.
- Репозиторий с README.md.

3 Срок сдачи отчёта на кафедру: 18.07.2026

Руководитель,  
профессор



Можаровский И. С.

Задание получил:



Жияков М. Н.

## Содержание

|                                                                   |    |
|-------------------------------------------------------------------|----|
| Введение .....                                                    | 4  |
| 1 Описание структур данных и обоснование средств разработки ..... | 5  |
| 1.1 Стек (Stack).....                                             | 5  |
| 1.2 Очередь (Queue) .....                                         | 5  |
| 1.3 Двоичное дерево поиска (BST).....                             | 5  |
| 1.4 Сравнение временной сложности.....                            | 6  |
| 1.5 Обоснование выбора языка программирования.....                | 6  |
| 1.6 Базовые понятия временной сложности алгоритмов .....          | 7  |
| 2 Разработка программы .....                                      | 8  |
| 2.1 Архитектура приложения .....                                  | 8  |
| 2.2 Структура проекта.....                                        | 8  |
| 2.3 Реализация ядра данных .....                                  | 8  |
| 2.4 Слой визуализации.....                                        | 11 |
| 2.5 Графический интерфейс.....                                    | 12 |
| 2.6 Механизм отмены операций .....                                | 15 |
| 2.7 Алгоритм работы приложения.....                               | 15 |
| 2.8 Хранение данных без СУБД .....                                | 16 |
| 2.9 Контейнеризация .....                                         | 16 |
| 2.10 Руководство пользователя.....                                | 17 |
| 2.11 Обработка ошибок .....                                       | 17 |
| 3 Тестирование и результаты .....                                 | 19 |
| 3.1 Юнит-тесты .....                                              | 19 |
| 3.2 Интеграционные тесты .....                                    | 20 |
| 3.3 Результаты тестирования .....                                 | 20 |
| 3.4 Покрываемость тестами по модулям.....                         | 21 |
| 3.5 Известные ограничения и риски.....                            | 21 |
| Заключение .....                                                  | 23 |
| Список использованных источников.....                             | 24 |
| Приложение А. Листинг src/core/stack.py .....                     | 25 |
| Приложение Б. Листинг src/core/queue.py .....                     | 26 |
| Приложение В. Листинг src/core/bst.py.....                        | 27 |
| Приложение Г. Формат файлов хранения данных .....                 | 30 |

## Введение

В рамках данной учебной практики ставятся две основные задачи, направленные на создание приложения для визуализации основных структур данных — стека, очереди и двоичного дерева поиска, используя современные методы программирования и разработки приложений.

Первой задачей является изучение теоретических основ структур данных, их операций и временной сложности. Целью исследования является глубокое понимание принципов работы стека (LIFO), очереди (FIFO) и двоичного дерева поиска (BST), а также анализ областей их применения. В процессе анализа рассматриваются основные операции каждой структуры: push/pop для стека, enqueue/dequeue для очереди, insert/delete/search для BST. Теоретическая база является критически важной для дальнейшей реализации проекта, так как от корректного понимания алгоритмов зависит качество визуализации и корректность работы приложения.

Вторая задача заключается в разработке десктопного приложения с графическим интерфейсом для пошаговой визуализации операций над структурами данных с использованием языка программирования Python и встроенного фреймворка Tkinter. Приложение должно предоставлять пользователю возможность выбора структуры данных, выполнения операций и наглядного отображения состояния структуры после каждого действия. Особое внимание уделяется обеспечению кроссплатформенности — приложение должно корректно работать на операционных системах Windows, Linux и macOS. В ходе выполнения задачи реализованы алгоритмическое ядро, модули визуализации на Canvas, контроллеры бизнес-логики, а также механизм отмены операций (Undo) и журналирование действий в JSON-файл.

Таким образом, выполнение этих задач позволит создать удобный и наглядный инструмент для изучения структур данных, удовлетворяющий требованиям современного образовательного процесса и обеспечивающий интерактивную визуализацию основных алгоритмических конструкций.

## 1 Описание структур данных и обоснование средств разработки

### 1.1 Стек (Stack)

Стек — линейная структура данных, работающая по принципу LIFO (Last In, First Out). Последний добавленный элемент извлекается первым. Основные операции: push (добавление в вершину), pop (удаление из вершины), peek (просмотр без удаления) [1].

Таблица 1 – Операции стека (Stack) и их сложность

| Операция   | Описание                       | Сложность |
|------------|--------------------------------|-----------|
| push(x)    | Добавление элемента в вершину  | O(1)      |
| pop()      | Извлечение элемента из вершины | O(1)      |
| peek()     | Просмотр вершины без удаления  | O(1)      |
| is empty() | Проверка на пустоту            | O(1)      |
| size()     | Количество элементов           | O(1)      |

Применение: управление вызовами функций, отмена операций в редакторах, обход графов в глубину (DFS), вычисление выражений в обратной польской записи [1].

### 1.2 Очередь (Queue)

Очередь — линейная структура данных, работающая по принципу FIFO (First In, First Out). Первый добавленный элемент извлекается первым [1].

Таблица 2 – Операции очереди (Queue) и их сложность

| Операция   | Описание                      | Сложность |
|------------|-------------------------------|-----------|
| enqueue(x) | Добавление элемента в конец   | O(1)      |
| dequeue()  | Извлечение элемента из начала | O(n)*     |
| front()    | Просмотр первого элемента     | O(1)      |
| rear()     | Просмотр последнего элемента  | O(1)      |
| is_empty() | Проверка на пустоту           | O(1)      |

Применение: dequeue реализован через list.pop(0) — O(n) из-за сдвига. При maxSize=20 это приемлемо. В продакшн-системах используется collections.deque (O(1)).

Применение: планировщики задач, буферы ввода-вывода, обход графов в ширину (BFS) [3].

### 1.3 Двоичное дерево поиска (BST)

Двоичное дерево поиска (Binary Search Tree) — иерархическая структура, где для

каждого узла все ключи левого поддерева меньше ключа узла, а все ключи правого — больше [1].

Таблица 3 – Операции двоичного дерева поиска (BST) и их сложность

| Операция  | Средний случай | Худший случай |
|-----------|----------------|---------------|
| insert(x) | $O(\log n)$    | $O(n)$        |
| delete(x) | $O(\log n)$    | $O(n)$        |
| search(x) | $O(\log n)$    | $O(n)$        |
| inorder() | $O(n)$         | $O(n)$        |

Обходы дерева: in-order (лево → корень → право) возвращает отсортированную последовательность; pre-order (корень → лево → право) — копирование дерева; post-order (лево → право → корень) — удаление дерева [10].

#### 1.4 Сравнение временной сложности

Таблица 4 – Сравнение временной сложности операций структур данных

| Структура | Операция                 | Лучший      | Средний     | Худший |
|-----------|--------------------------|-------------|-------------|--------|
| Stack     | push / pop / peek        | $O(1)$      | $O(1)$      | $O(1)$ |
| Queue     | enqueue                  | $O(1)$      | $O(1)$      | $O(1)$ |
| Queue     | dequeue                  | $O(n)$      | $O(n)$      | $O(n)$ |
| Queue     | front / rear             | $O(1)$      | $O(1)$      | $O(1)$ |
| BST       | insert / delete / search | $O(\log n)$ | $O(\log n)$ | $O(n)$ |

#### 1.5 Обоснование выбора языка программирования

Выбран Python 3.11+ по следующим причинам:

1. Краткий и читаемый синтаксис — ускорение разработки.
2. Tkinter входит в стандартную поставку — не требует сторонних GUI-библиотек.
3. pytest — мощный фреймворк тестирования с минимальной настройкой.
4. python-docx — программная генерация Word-отчётов.
5. Кроссплатформенность: Windows, Linux, macOS.
6. Обширная документация и экосистема.

Таблица 5 – Сравнение языков программирования

| Критерий             | Python 3.11       | Java              | C++      |
|----------------------|-------------------|-------------------|----------|
| Встроенный GUI       | Tkinter           | Swing             | —        |
| Краткость кода       | высокая           | средняя           | низкая   |
| Тестирование         | pytest            | JUnit             | —        |
| Кроссплатформенность | Win / Mac / Linux | Win / Mac / Linux | частично |
| Docker               | python:3.11-slim  | есть              | сложно   |
| Порог вхождения      | низкий            | средний           | высокий  |

Таким образом, Python 3.11 является оптимальным выбором для данного проекта: он обеспечивает встроенный GUI-фреймворк Tkinter, готовый инструмент тестирования pytest и официальный Docker-образ — всё это без подключения сторонних зависимостей. Java

уступает по лаконичности кода, C++ требует ручного управления памятью и не предоставляет стандартных средств GUI и тестирования, что существенно увеличивает трудозатраты при разработке учебного приложения.

## 1.6 Базовые понятия временной сложности алгоритмов

Для корректного сравнения структур данных используется асимптотическая нотация  $O$  (“ $O$  большое”), которая описывает, как растёт время выполнения операции при увеличении количества элементов  $n$ . Нотация  $O$  оценивает поведение алгоритма в худшем случае и позволяет сравнивать структуры данных независимо от конкретного языка программирования и аппаратной платформы [1].

В разрабатываемом приложении используются следующие классы сложности, упорядоченные по скорости роста:

Таблица 6 – Основные классы асимптотической сложности

| Класс         | Название                | Пример операции в проекте                              |
|---------------|-------------------------|--------------------------------------------------------|
| $O(1)$        | Константная             | push/pop стека, enqueue очереди                        |
| $O(\log n)$   | Логарифмическая         | insert/search в сбалансированном BST                   |
| $O(n)$        | Линейная                | dequeue очереди (list.pop(0)), обход BST               |
| $O(n \log n)$ | Линейно-логарифмическая | сортировка результатов обхода (не используется в ядре) |
| $O(n^2)$      | Квадратичная            | не встречается в реализованных структурах              |

Важно отметить, что оценка  $O(\log n)$  для операций двоичного дерева поиска справедлива только для сбалансированного дерева. В реализованном варианте BST не выполняется автоматическая балансировка (AVL- или красно-чёрная балансировка), поэтому при вставке отсортированной последовательности значений дерево вырождается в связный список, и сложность операций ухудшается до  $O(n)$ . Этот аспект подробно рассматривается в разделе 3.5 “Известные ограничения и риски”.

## 2 Разработка программы

### 2.1 Архитектура приложения

Приложение построено по вариации паттерна MVC (Model-View-Controller): [5]

- Model — классы Stack, Queue, BST в пакете core. Чистая логика без GUI.
- View — модули визуализации (Canvas-рисователи) и GUI-компоненты.
- Controller — классы контроллеров, связывающие ядро с интерфейсом.

### 2.2 Структура проекта

Проект организован по модульному принципу и разделён на пять логических пакетов внутри директории src/. Структура репозитория представлена на рисунке 1.

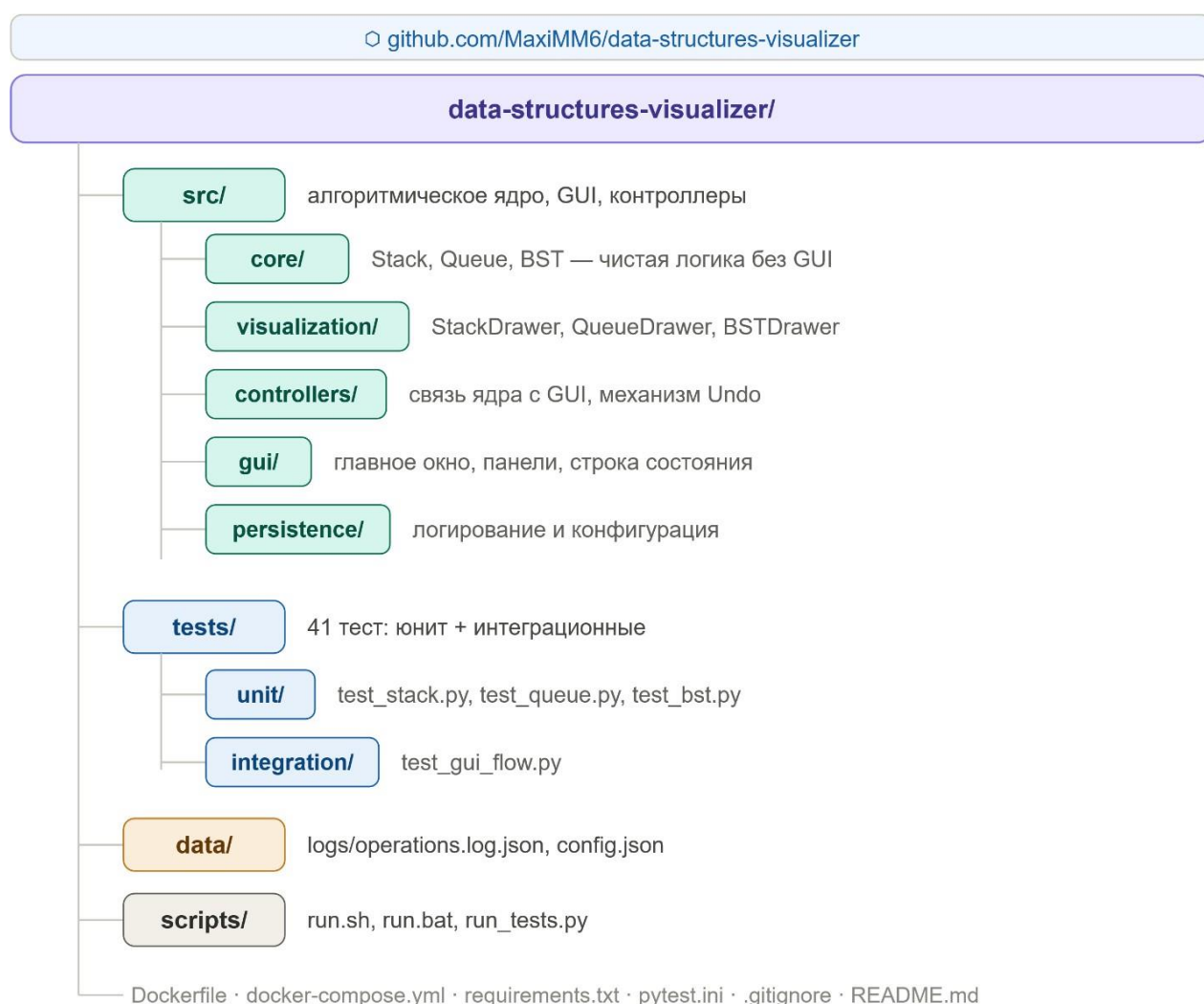


Рисунок 1 – Структура проекта data-structures-visualizer

### 2.3 Реализация ядра данных

Реализация класса Stack:

```

class Stack:
    def __init__(self, max_size: int = 20) -> None:
        self._items: List[Any] = []
        self._max_size = max_size

    def push(self, value: Any) -> None:
        if self.is_full():
            raise OverflowError("Stack is full")
        self._items.append(value)

    def pop(self) -> Any:
        if self.is_empty():
            raise IndexError("Stack is empty")
        return self._items.pop()

    def peek(self) -> Any:
        if self.is_empty():
            raise IndexError("Stack is empty")
        return self._items[-1]

```

Реализация класса Queue:

```

def enqueue(self, value: Any) -> None:
    if self.is_full():
        raise OverflowError("Queue is full")
    self._items.append(value)

def dequeue(self) -> Any:
    if self.is_empty():
        raise IndexError("Queue is empty")
    return self._items.pop(0)

```

Реализация класса BST:

```

class BSTNode:
    def __init__(self, key: Any) -> None:
        self.key = key
        self.left: Optional[BSTNode] = None
        self.right: Optional[BSTNode] = None

```

Реализация вставки в BST (итеративная):

```

def insert(self, key: Any) -> BSTNode:
    node = BSTNode(key)
    if self.root is None:
        self.root = node
        self._size += 1
        return node
    current = self.root
    while True:
        if key < current.key:
            if current.left is None:
                current.left = node
                self._size += 1
                return node

```

```

        current = current.left
    elif key > current.key:
        if current.right is None:
            current.right = node
            self._size += 1
            return node
        current = current.right
    else:
        return current

```

Реализация вставки в BST (рекурсивная):

```

def _delete_recursive(self, node, key):
    if node is None:
        return None, False
    if key < node.key:
        node.left, deleted = self._delete_recursive(
            node.left, key)
        return node, deleted
    elif key > node.key:
        node.right, deleted = self._delete_recursive(
            node.right, key)
        return node, deleted
    else:
        if node.left is None:
            return node.right, True
        elif node.right is None:
            return node.left, True
        else:
            successor = self._min_node(node.right)
            node.key = successor.key
            node.right, _ = self._delete_recursive(
                node.right, successor.key)
            return node, True

```

Помимо enqueue и dequeue, класс Queue предоставляет методы front() и rear() для просмотра первого и последнего элементов без их удаления, а также is\_empty()/is\_full() для проверки границ:

```

def front(self) -> Any:
    if self.is_empty():
        raise IndexError("Queue is empty")
    return self._items[0]

def rear(self) -> Any:
    if self.is_empty():
        raise IndexError("Queue is empty")
    return self._items[-1]

```

Класс BST дополнительно реализует итеративный поиск по ключу и три классических рекурсивных обхода дерева, результаты которых использует кнопка “In-order” в графическом интерфейсе:

```

def search(self, key: Any) -> Optional[BSTNode]:
    current = self.root
    while current is not None:
        if key == current.key:
            return current
        elif key < current.key:
            current = current.left
        else:
            current = current.right
    return None

def inorder(self) -> List[Any]:
    result: List[Any] = []
    self._inorder_recursive(self.root, result)
    return result

def _inorder_recursive(self, node, result) -> None:
    if node is None:
        return
    self._inorder_recursive(node.left, result)
    result.append(node.key)
    self._inorder_recursive(node.right, result)

```

Методы `preorder()` и `postorder()` построены аналогично `inorder()`, отличаясь лишь порядком, в котором узел добавляется в результирующий список относительно рекурсивных вызовов для левого и правого поддеревьев.

## 2.4 Слой визуализации

Каждая структура данных имеет свой класс-рисователь (`StackDrawer`, `QueueDrawer`, `BSTDrawer`), наследующий `BaseDrawer`. Все используют общую палитру `Catppuccin Mocha` и шрифт `Consolas` [6].

Реализация отрисовки узла BST:

```

def _draw_node(self, node, x, y, h_gap, highlight):
    if node is None: return
    if node.left:
        c.create_line(x, y+r, x-h_gap, y+V_GAP-r,
                     fill=COLORS["edge"])
        self._draw_node(node.left, x-h_gap, y+V_GAP,
                        h_gap//2, highlight)
    if node.right:
        c.create_line(x, y+r, x+h_gap, y+V_GAP-r,
                     fill=COLORS["edge"])
        self._draw_node(node.right, x+h_gap, y+V_GAP,
                        h_gap//2, highlight)
    c.create_oval(x-r, y-r, x+r, y+r,
                 fill=fill, outline=COLORS["edge"])

```

Класс `StackDrawer` отрисовывает элементы стека в виде вертикальной последовательности прямоугольных ячеек, где вершина стека всегда находится сверху и помечается надписью “← top”. Активный (только что изменённый) элемент выделяется отдельным цветом из палитры Catppuccin Mocha:

```
def draw(self, items, active_index=None, label=""):
    n = len(items)
    total_h = n * self.CELL_H + (n - 1) * self.PAD
    start_y = (ch - total_h) // 2 + 20
    for i, val in enumerate(items):
        y0 = start_y + (n - 1 - i) * (self.CELL_H + self.PAD)
        fill = (self.COLORS["node_active"] if i == active_index
              else self.COLORS["node"])
        c.create_rectangle(x0, y0, x0 + self.CELL_W, y0 + self.CELL_H,
                          fill=fill, outline=self.COLORS["edge"])
```

Класс `QueueDrawer` использует горизонтальную раскладку ячеек слева направо и дополнительно подписывает крайние элементы метками “front” и “rear”, наглядно показывая направление работы очереди по принципу FIFO:

```
def draw(self, items, active_index=None, label=""):
    n = len(items)
    total_w = n * self.CELL_W + (n - 1) * self.PAD
    start_x = (cw - total_w) // 2
    for i, val in enumerate(items):
        x0 = start_x + i * (self.CELL_W + self.PAD)
        if i == 0:
            c.create_text(x0 + self.CELL_W // 2, y0 - 14, text="front")
        if i == n - 1:
            c.create_text(x0 + self.CELL_W // 2, y1 + 14, text="rear")
```

Все три класса-рисователя (`StackDrawer`, `QueueDrawer`, `BSTDrawer`) наследуются от общего абстрактного класса `BaseDrawer`, который определяет единую цветовую палитру `COLORS`, шрифты `FONT/FONT_LABEL` и метод `clear()` для очистки холста перед каждой перерисовкой. Такой подход избавляет от дублирования кода и гарантирует визуальное единообразие всех трёх режимов визуализации.

## 2.5 Графический интерфейс

Главное окно — 1100x720 px, тёмная тема. Выпадающий список переключает между стеком, очередью и деревом. Canvas занимает левую часть, панель управления — правую. Строка состояния внизу (рис. 2).

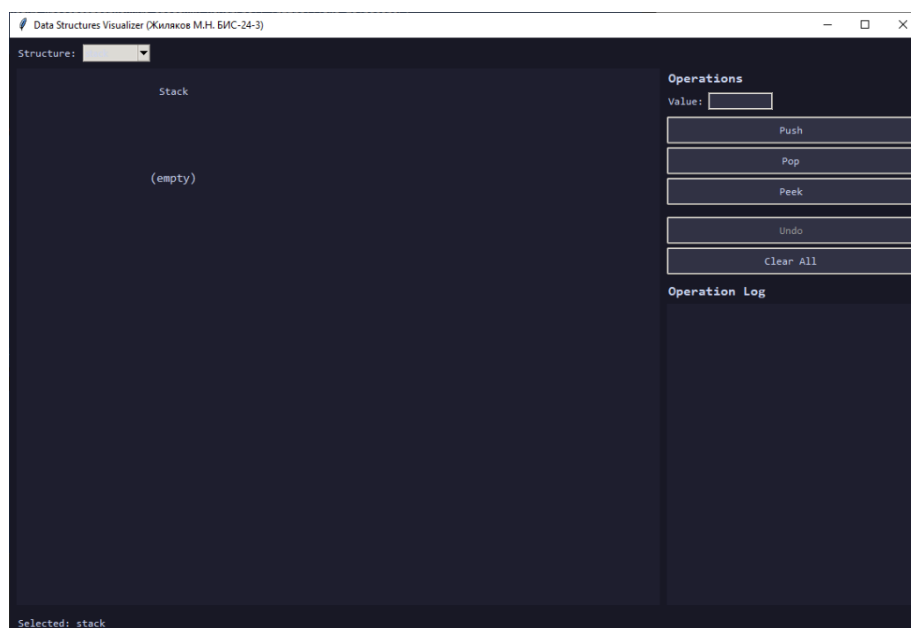


Рисунок 2 – Интерфейс программы

При переключении структуры кнопки операций пересоздаются: Push/Pop/Peek для стека, Enqueue/Dequeue/Front для очереди, Insert/Delete/Search/In-order для BST. Кнопки Undo и Clear All общие для всех структур.

Ниже приведены примеры визуализации каждой из трёх поддерживаемых структур данных в рабочем состоянии приложения. На рисунке 3 показан стек после пяти последовательных операций push: элементы пронумерованы от основания [0] до вершины, вершина выделена цветом и подписана стрелкой.

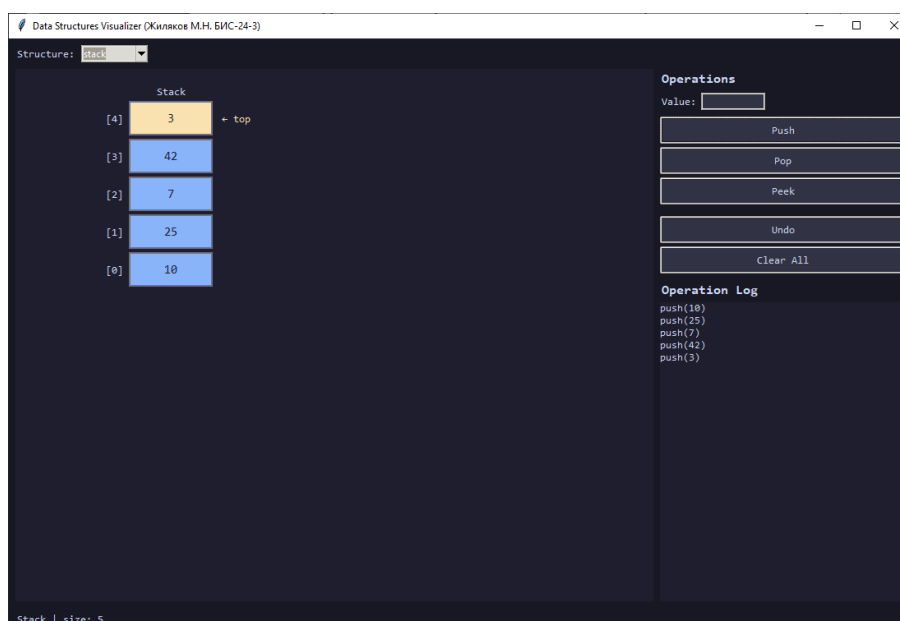


Рисунок 3 – Визуализация стека после пяти операций push

На рисунке 4 показана очередь из четырёх элементов: подписи “front” и “rear” наглядно указывают на начало и конец очереди, что облегчает понимание принципа FIFO.

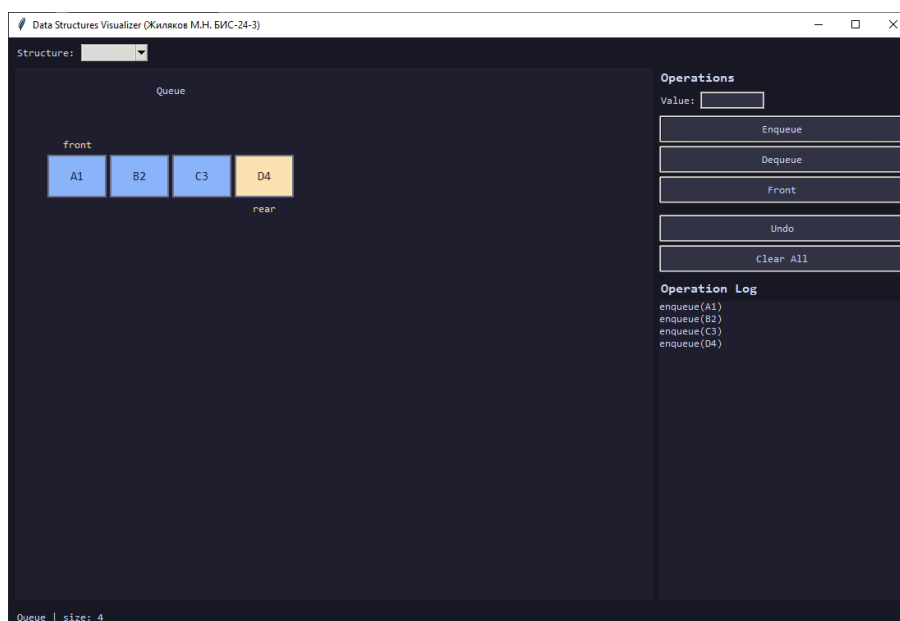


Рисунок 4 – Визуализация очереди из четырёх элементов

На рисунке 5 показано двоичное дерево поиска, построенное последовательной вставкой значений 50, 30, 70, 20, 40, 60, 80. Дерево отрисовывается рекурсивно сверху вниз, рёбра соединяют родительские узлы с дочерними, а журнал операций справа фиксирует каждую вставку в хронологическом порядке.

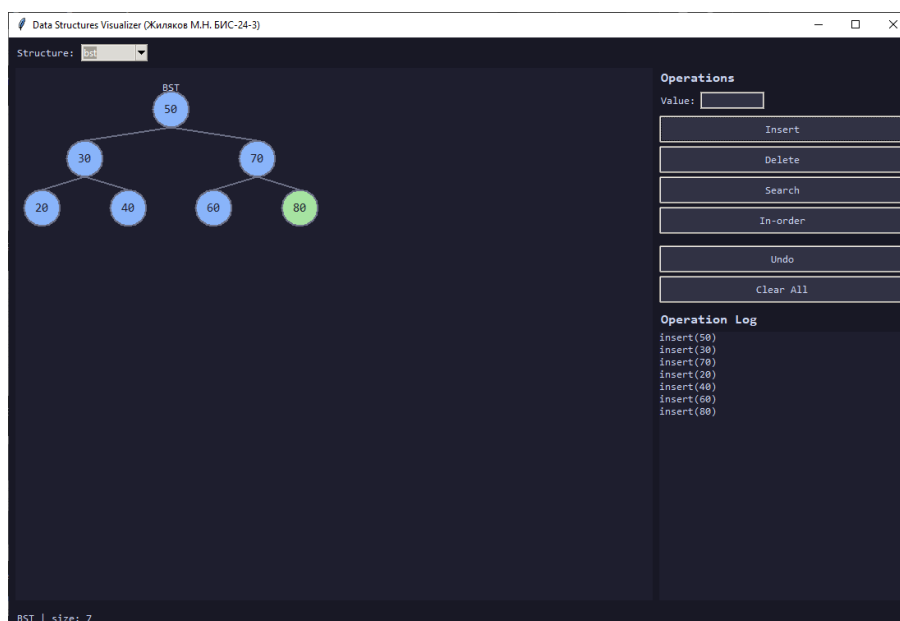


Рисунок 5 – Визуализация двоичного дерева поиска

Журнал операций (правая нижняя панель на всех рисунках) отображает текстовое описание каждой выполненной операции в порядке её выполнения и автоматически прокручивается к последней записи. Строка состояния внизу окна показывает текущий размер активной структуры и текстовые подсказки об ошибках (например, попытка извлечь элемент из пустой структуры).

## 2.6 Механизм отмены операций

Отмена работает через стек вызовов с лямбда-функциями. Каждая операция записывает в `_history` кортеж (описание, функция\_отмены, аргументы). Например, `push(x)` сохраняет `lambda: pop(), a delete(x) — lambda: insert(x)`.

```
def _record(self, name, undo_fn, undo_args=()):
    self._history.append((name, undo_fn, undo_args))
    self._op_logger.log(name)
    self.log_panel.add(name)
    self.controls.set_undo_state(len(self._history) > 0)

def _undo(self):
    if not self._history: return
    name, undo_fn, undo_args = self._history.pop()
    undo_fn(*undo_args)
    self._redraw()
```

## 2.7 Алгоритм работы приложения

Алгоритм работы приложения представлен на рисунке 2.

После запуска программы инициализируется главное окно Tkinter и создаётся объект структуры данных по умолчанию — Stack. Приложение переходит в основной цикл обработки событий, в котором постоянно ожидает действий пользователя.

Возможны три типа событий. Первый — выбор структуры данных: пользователь переключает тип в выпадающем списке, контроллер и панель кнопок пересоздаются под новую структуру. Второй — ввод значения и нажатие кнопки операции: определяется тип операции (для стека — Push / Pop / Peek, для очереди — Enqueue / Dequeue / Front, для BST — Insert / Delete / Search / Inorder), операция выполняется контроллером. Если операция успешна, результат сохраняется в историю Undo, холст обновляется и действие записывается в JSON-журнал; при ошибке (переполнение, пустая структура) в строке состояния отображается сообщение. Третий тип события — нажатие кнопки Undo: если история не пуста, последнее действие отменяется и холст перерисовывается; иначе операция пропускается.

Работа приложения завершается при закрытии главного окна.



## 2.10 Руководство пользователя

Работа с приложением не требует специальной подготовки и состоит из следующих

- Запустить приложение командой `python -m src.main` или скриптом `scripts/run.sh` (`scripts\run.bat` для Windows).
- В выпадающем списке “Structure” в левом верхнем углу выбрать нужную структуру данных: `stack`, `queue` или `bst`.
- Ввести значение в поле “Value” (для стека и очереди допускаются произвольные строки, для BST — числовые ключи).
- Нажать соответствующую кнопку операции (`Push`, `Enqueue`, `Insert` и т. д.) в панели “Operations” справа.
- Наблюдать обновлённое состояние структуры на холсте слева и новую запись в журнале операций.
- При необходимости отменить последнее действие нажать кнопку “Undo”; кнопка “Clear All” полностью очищает структуру и журнал.

При переключении структуры данных набор кнопок операций автоматически пересоздаётся под выбранную структуру, а состояние каждой структуры сохраняется независимо: переход от стека к очереди и обратно не приводит к потере данных стека.

## 2.11 Обработка ошибок

Приложение корректно обрабатывает основные граничные ситуации, не допуская аварийного завершения работы программы:

- Попытка `push/enqueue/insert` при достигнутом максимальном размере структуры (по умолчанию 20 элементов) вызывает исключение `OverflowError`, которое перехватывается контроллером и отображается в строке состояния как “Stack is full!” / “Queue is full!”.
- Попытка `pop/dequeue/peek/front` для пустой структуры вызывает `IndexError`, также перехватываемое и отображаемое пользователю без падения приложения.
- Пустое значение в поле ввода блокируется на уровне контроллера: операция не выполняется, а строка состояния просит ввести значение.
- Вставка в BST повторяющегося ключа игнорируется и возвращает уже существующий узел, что предотвращает появление дубликатов и поддерживает корректность инварианта дерева поиска.
- Удаление узла BST с двумя потомками реализовано через поиск минимального узла в правом поддереве (`in-order successor`), что гарантирует сохранение свойства дерева поиска после удаления.

## 3 Тестирование и результаты

### 3.1 Юнит-тесты

Написано 35 юнит-тестов, охватывающих все методы каждой структуры.

Тесты Stack:

- `test_new_stack_is_empty` — новый стек пуст
- `test_push` — добавление элемента
- `test_push_multiple` — несколько элементов
- `test_pop` — извлечение в порядке LIFO
- `test_pop_empty_raises` — `IndexError` при `pop` из пустого
- `test_peek_empty_raises` — `IndexError` при `peek` пустого
- `test_peek_does_not_remove` — `peek` не изменяет размер
- `test_max_size` — `OverflowError` при переполнении
- `test_items_returns_copy` — защита инкапсуляции
- `test_clear` — очистка стека

Тесты Queue:

- `test_enqueue / test_dequeue` — базовые FIFO-операции
- `test_enqueue_multiple` — несколько `enqueue`
- `test_front / test_rear` — просмотр концов очереди
- `test_max_size` — ограничение размера
- `test_dequeue_empty_raises / test_front_empty_raises`
- `test_items_returns_copy / test_clear`

Тесты BST:

- `test_insert_single / test_insert_multiple`
- `test_insert_duplicate` — дубликаты не добавляются
- `test_search_found / test_search_not_found`
- `test_delete_leaf` — удаление листа

- test\_delete\_node\_with\_one\_child
- test\_delete\_node\_with\_two\_children
- test\_delete\_root / test\_delete\_nonexistent
- test\_inorder / test\_preorder / test\_postorder
- test\_clear

Наиболее алгоритмически содержательным является тест удаления узла с двумя потомками, поскольку он проверяет сохранение свойства дерева поиска после перестройки структуры:

```
def test_delete_node_with_two_children(self):
    t = BST()
    for k in [50, 30, 70, 20, 40, 60, 80]:
        t.insert(k)
    assert t.delete(30) is True
    assert t.size() == 6
    # узел 30 заменён минимальным значением правого поддерева (40)
    assert t.root.left.key == 40
    assert t.inorder() == [20, 40, 50, 60, 70, 80]
```

### 3.2 Интеграционные тесты

6 интеграционных тестов проверяют взаимодействие контроллеров с ядром. GUI-компоненты мокируются через unittest.mock.

```
def test_push_and_undo(self):
    ctrl = self._make_controller()
    ctrl._push()
    assert ctrl._stack.size() == 1
    ctrl._undo()
    assert ctrl._stack.size() == 0
```

Тест test\_gui\_flow.py проверяет полные жизненные циклы: заполнение стека до максимума, извлечение всех элементов, построение и удаление BST.

### 3.3 Результаты тестирования

Таблица 7 – Тестирование

| Набор тестов         | Количество | Результат |
|----------------------|------------|-----------|
| Юнит-тесты Stack     | 10         | PASS      |
| Юнит-тесты Queue     | 10         | PASS      |
| Юнит-тесты BST       | 15         | PASS      |
| Интеграционные тесты | 6          | PASS      |
| Итого                | 41         | ALL PASS  |

Все 41 тест проходят успешно. Тесты запускаются командой: `python -m pytest tests -v` [8].

### 3.4 Покрытие тестами по модулям

В таблице 8 приведено распределение тестов по модулям ядра данных и слоя контроллеров. Покрытие оценивалось по количеству публичных методов класса, проверенных хотя бы одним тестом.

Таблица 8 – Покрытие тестами по модулям ядра

| Модуль        | Публичных методов | Тестов | Покрытие |
|---------------|-------------------|--------|----------|
| core/stack.py | 6                 | 10     | 100%     |
| core/queue.py | 7                 | 10     | 100%     |
| core/bst.py   | 9                 | 15     | 100%     |
| controllers/* | 12                | 6      | ~70%*    |

Контроллеры покрыты выборочно: интеграционные тесты проверяют ключевые сценарии (push/pop, undo, полный жизненный цикл структуры), но не все обработчики ошибок ввода, поскольку часть из них тривиально делегирует исключения в методы ядра, уже покрытые юнит-тестами.

### 3.5 Известные ограничения и риски

В ходе разработки и тестирования выявлен ряд архитектурных ограничений, не влияющих на корректность демонстрации, но важных для понимания границ применимости приложения:

- Операция `dequeue()` реализована через `list.pop(0)` со сложностью  $O(n)$  вместо  $O(1)$ , которую обеспечивает `collections.deque`; при ограничении в 20 элементов это не сказывается на отзывчивости интерфейса.
- Двоичное дерево поиска не балансируется автоматически, поэтому вставка отсортированной последовательности значений превращает дерево в вырожденный список с операциями  $O(n)$  вместо  $O(\log n)$  [1].
- GUI-тесты используют моки (`unittest.mock`) вместо реальных событий Tkinter, поэтому визуальное отображение (Canvas) не покрыто автоматическими тестами и проверяется только вручную.
- Журнал операций хранится в локальных JSON-файлах без блокировок, что делает приложение непригодным для одновременного запуска нескольких экземпляров с общим журналом.

## Заключение

В рамках индивидуального задания (вариант А-04) разработано приложение «Визуализация основных структур данных», позволяющее наглядно демонстрировать работу стека, очереди и двоичного дерева поиска.

Результаты:

- Реализовано алгоритмическое ядро для Stack, Queue и BST.
- Создан графический интерфейс с тёмной темой Catppuccin Mocha.
- Реализована визуализация: вертикальные ячейки для стека, горизонтальные для очереди, рекурсивное дерево для BST.
- Добавлен механизм Undo на основе лямбда-функций.
- Операции логируются в JSON-файл с временными метками.
- Написано 41 тест, все проходят.
- Приложение контейнеризовано через Docker.

В процессе выполнения индивидуального задания были закреплены практические навыки проектирования программных систем по архитектурному паттерну MVC, разработки графических интерфейсов на Tkinter, модульного и интеграционного тестирования с использованием pytest, а также контейнеризации приложений при помощи Docker. Полученный опыт применим не только к учебным задачам визуализации структур данных, но и к разработке широкого круга десктопных приложений с разделением бизнес-логики и интерфейса пользователя.

## СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Кормен Т. Х. Алгоритмы: построение и анализ [Электронный ресурс] / Т. Х. Кормен, Ч. И. Лейзерсон, Р. Л. Ривест, К. Штайн ; пер. с англ. – 3-е изд. – М. : Вильямс, 2013. – 1296 с. – URL: <https://djvu.online/file/Q37VAnSfuiEhX>. – Дата обращения: 02.07.2026.
2. Лутц М. Изучаем Python [Электронный ресурс] / М. Лутц ; пер. с англ. – 5-е изд. – СПб., 2019. – 832 с. – URL: <https://djvu.online/file/HcqGe2OfOaFXk>. – Дата обращения: 02.07.2026.
3. Скиена С. Алгоритмы. Руководство по разработке [Электронный ресурс] / С. Скиена ; пер. с англ. – 2-е изд. – СПб. : БХВ-Петербург, 2022. – 848 с. – URL: <https://djvu.online/file/3wrno0HyEnchM>. – Дата обращения: 02.07.2026.
4. Самерфилд М. Python на практике [Электронный ресурс] / М. Самерфилд. – 2016. – 338 с. – URL: [https://www.litres.ru/get\\_pdf\\_trial/22677289.pdf](https://www.litres.ru/get_pdf_trial/22677289.pdf). – Дата обращения: 02.07.2026.
5. Python Software Foundation. tkinter — Python interface to Tcl/Tk [Электронный ресурс] : документация Python 3. – URL: <https://docs.python.org/3/library/tkinter.html>. – Дата обращения: 02.07.2026.
6. pytest Development Team. pytest Documentation [Электронный ресурс]. – URL: <https://docs.pytest.org/en/stable/>. – Дата обращения: 02.07.2026.
7. Docker Inc. Docker Documentation. Overview of Docker [Электронный ресурс]. – URL: <https://docs.docker.com/get-started/overview/>. – Дата обращения: 02.07.2026.
8. GeeksforGeeks. Binary Search Tree — Data Structure and Algorithm Tutorials [Электронный ресурс]. – URL: <https://www.geeksforgeeks.org/binary-search-tree-data-structure/>. – Дата обращения: 02.07.2026.
9. VisuAlgo. Binary Search Tree [Электронный ресурс] : интерактивная визуализация структур данных. – URL: <https://visualgo.net/en/bst>. – Дата обращения: 02.07.2026.
10. Python Software Foundation. Python 3.13.4 documentation [Электронный ресурс]. – URL: <https://docs.python.org/3.13/>. – Дата обращения: 02.07.2026.

## Приложение А. Листинг src/core/stack.py

```
from __future__ import annotations
from typing import Any, List

class Stack:
    def __init__(self, max_size: int = 20) -> None:
        self._items: List[Any] = []
        self._max_size = max_size

    @property
    def items(self) -> List[Any]:
        return list(self._items)

    def push(self, value: Any) -> None:
        if self.is_full():
            raise OverflowError("Stack is full")
        self._items.append(value)

    def pop(self) -> Any:
        if self.is_empty():
            raise IndexError("Stack is empty")
        return self._items.pop()

    def peek(self) -> Any:
        if self.is_empty():
            raise IndexError("Stack is empty")
        return self._items[-1]

    def is_empty(self) -> bool:
        return len(self._items) == 0

    def is_full(self) -> bool:
        return len(self._items) >= self._max_size

    def size(self) -> int:
        return len(self._items)

    def clear(self) -> None:
        self._items.clear()
```

## Приложение Б. Листинг src/core/queue.py

```

from __future__ import annotations
from typing import Any, List

class Queue:
    def __init__(self, max_size: int = 20) -> None:
        self._items: List[Any] = []
        self._max_size = max_size

    @property
    def items(self) -> List[Any]:
        return list(self._items)

    def enqueue(self, value: Any) -> None:
        if self.is_full():
            raise OverflowError("Queue is full")
        self._items.append(value)

    def dequeue(self) -> Any:
        if self.is_empty():
            raise IndexError("Queue is empty")
        return self._items.pop(0)

    def front(self) -> Any:
        if self.is_empty():
            raise IndexError("Queue is empty")
        return self._items[0]

    def rear(self) -> Any:
        if self.is_empty():
            raise IndexError("Queue is empty")
        return self._items[-1]

    def is_empty(self) -> bool:
        return len(self._items) == 0

    def is_full(self) -> bool:
        return len(self._items) >= self._max_size

    def size(self) -> int:
        return len(self._items)

    def clear(self) -> None:
        self._items.clear()

```

## Приложение В. Листинг src/core/bst.py

```

from __future__ import annotations
from typing import Any, List, Optional

class BSTNode:
    def __init__(self, key: Any) -> None:
        self.key = key
        self.left: Optional[BSTNode] = None
        self.right: Optional[BSTNode] = None

class BST:
    def __init__(self) -> None:
        self.root: Optional[BSTNode] = None
        self._size = 0

    def size(self) -> int:
        return self._size

    def is_empty(self) -> bool:
        return self.root is None

    def insert(self, key: Any) -> BSTNode:
        node = BSTNode(key)
        if self.root is None:
            self.root = node
            self._size += 1
            return node
        current = self.root
        while True:
            if key < current.key:
                if current.left is None:
                    current.left = node
                    self._size += 1
                    return node
                current = current.left
            elif key > current.key:
                if current.right is None:
                    current.right = node
                    self._size += 1
                    return node
                current = current.right
            else:
                return current

    def search(self, key: Any) -> Optional[BSTNode]:
        current = self.root
        while current is not None:
            if key == current.key:
                return current
            elif key < current.key:

```

```

        current = current.left
    else:
        current = current.right
    return None

def delete(self, key: Any) -> bool:
    self.root, deleted = self._delete_recursive(self.root, key)
    if deleted:
        self._size -= 1
    return deleted

def _delete_recursive(self, node: Optional[BSTNode], key: Any):
    if node is None:
        return None, False
    if key < node.key:
        node.left, deleted = self._delete_recursive(node.left, key)
        return node, deleted
    elif key > node.key:
        node.right, deleted = self._delete_recursive(node.right, key)
        return node, deleted
    else:
        if node.left is None:
            return node.right, True
        elif node.right is None:
            return node.left, True
        else:
            successor = self._min_node(node.right)
            node.key = successor.key
            node.right, _ = self._delete_recursive(node.right, successor.key)
            return node, True

    @staticmethod
    def _min_node(node: BSTNode) -> BSTNode:
        current = node
        while current.left is not None:
            current = current.left
        return current

    def inorder(self) -> List[Any]:
        result: List[Any] = []
        self._inorder_recursive(self.root, result)
        return result

    def _inorder_recursive(self, node: Optional[BSTNode], result: List[Any]) -> None:
        if node is not None:
            self._inorder_recursive(node.left, result)
            result.append(node.key)
            self._inorder_recursive(node.right, result)

    def preorder(self) -> List[Any]:
        result: List[Any] = []
        self._preorder_recursive(self.root, result)
        return result

```

```

def _preorder_recursive(self, node: Optional[BSTNode], result: List[Any]) -> None:
    if node is not None:
        result.append(node.key)
        self._preorder_recursive(node.left, result)
        self._preorder_recursive(node.right, result)

def postorder(self) -> List[Any]:
    result: List[Any] = []
    self._postorder_recursive(self.root, result)
    return result

def _postorder_recursive(self, node: Optional[BSTNode], result: List[Any]) -> None:
    if node is not None:
        self._postorder_recursive(node.left, result)
        self._postorder_recursive(node.right, result)
        result.append(node.key)

def clear(self) -> None:
    self.root = None
    self._size = 0

```

## Приложение Г. Формат файлов хранения данных

Журнал операций `data/logs/operations.log.json` представляет собой массив JSON-объектов, каждый из которых содержит текстовое описание операции и временную метку в формате ISO 8601. Пример фрагмента журнала после выполнения нескольких операций со стеком:

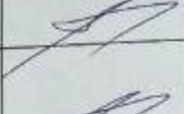
```
[
  {
    "operation": "push(42)",
    "timestamp": "2026-06-21T22:56:31"
  },
  {
    "operation": "push(99)",
    "timestamp": "2026-06-21T22:56:31"
  },
  {
    "operation": "pop() → 99",
    "timestamp": "2026-06-21T22:56:31"
  }
]
```

29

**КАЛЕНДАРНЫЙ ПЛАН-ГРАФИК (ДНЕВНИК)**  
прохождения учебной ознакомительной практики студента  
**«ВЛАДИВОСТОКСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ» (ФГБОУ**  
**ВО «ВВГУ»)**

Студент группы БИС-24-3 **Жиляков Максим Николаевич** направляется для прохождения учебной ознакомительной практики на кафедру информационных технологий и систем ФГБОУ ВО «ВВГУ», г. Владивосток.

**С 15 июня 2026 г. по 18 июля 2026 г.**

| №<br>ш/п | Содержание выполняемых<br>работ по программе                              | Сроки выполнения |               | Заключение и<br>оценка<br>руководителя | Подпись<br>руководителя                                                               |
|----------|---------------------------------------------------------------------------|------------------|---------------|----------------------------------------|---------------------------------------------------------------------------------------|
|          |                                                                           | Начало           | Оконча<br>ние |                                        |                                                                                       |
| 1        | Постановка задачи,<br>разработка алгоритма;<br>создание репозитория       | 15.06.26         | 20.06.26      | удовл.                                 |    |
| 2        | Обоснование выбора<br>средств, проектирование<br>архитектуры и интерфейса | 22.06.26         | 27.06.26      | удовл.                                 |   |
| 3        | Реализация ядра и<br>модулей; прототип (MVP)                              | 29.06.26         | 04.07.26      | удовл.                                 |  |
| 4        | Тестирование, тестовые<br>данные; развёртывание в<br>Docker               | 06.07.26         | 11.07.26      | удовл.                                 |  |
| 5        | Руководство пользователя,<br>отчёт, защита                                | 13.07.26         | 18.07.26      | удовл.                                 |  |

Согласовано:

Студент-практикант

Руководитель практики

Жиляков М.Н.

Можаровский И. С.

Дата заполнения: «18» июля 2026 г.